

ELI MKXI — Full Project Breakdown and Assessment

Contents

1. What ELI is	1
2. Scale & shape	1
3. Architecture, layer by layer	2
4. What's genuinely strong	2
5. Where it's weak (grounded in the sweep)	3
6. Honest verdict on "frontier, ground-breaking"	3
7. Highest-leverage work (the next month)	3

A grounded, total-project read: what ELI is, its scale and shape, the architecture layer by layer, what's strong, where it's weak (with numbers), an honest verdict, and the highest-leverage work. Companion to `orchestration_and_agents.md` (agent/bus detail) and `agent_bus.md`.

Method note: this is built from deep reads of the core modules (engine, executor, router, `agent_bus`, orchestrator, planners, memory, grounding gate, vision, identity, security, reasoning modes) plus a complete structural and code-health sweep (LOC distribution, debt markers, duplication, repo hygiene). It is grounded in a deep read of every package this session — claims are grounded in what was actually inspected.

1. What ELI is

A **local-first, model-agnostic personal AI** — no cloud, no telemetry, offline-by-default and hard-gated at the socket boundary (internet is an owner-controlled, monitored opt-in, not a runtime dependency; one-time opt-in HuggingFace model downloads are the only network event unless the owner enables the Internet toggle). It bundles: GGUF inference (llama-cpp), a PySide6 desktop GUI, persistent SQLite+FTS5+FAISS memory, a knowledge graph, a 12-stage cognitive pipeline with a parallel multi-agent bus, a deterministic grounding/evidence layer to fight confabulation, local vision (Qwen2.5-VL hot-swap + Moondream co-resident), TTS/STT, OS control, a plugin system, a proactive daemon, and a LoRA self-training loop.

As of 2026-06-28 it has a second front end: a self-hosted **FastAPI web app + dashboard PWA** (`api/server.py`) — chat, a live telemetry dashboard, **ELI's own MQTT smart-home** (rooms, scenes, real automations; Home Assistant removed), multi-user **RBAC** (admin/member/viewer), a **tamper-evident hash-chained audit trail**, collaborative **research corpora**, **browser voice** (local whisper STT + Piper TTS), and the monitored Internet toggle — launchable in-process from the desktop GUI. It is a cognitive operating system for one machine (and, optionally, the people the owner shares it with), not a chat wrapper.

2. Scale & shape

143,432 LOC across 370 Python files (`eli/`), plus the FastAPI web server (`api/server.py`, ~3,884 lines with an embedded dashboard PWA) and 194 test files. | Subsystem | LOC | Files | Role | |:-----|:-----|:-----|:-----|
| execution/ | 23.1k | 14 | router + executor (action dispatch) |
| runtime/ | 22.6k | 80 | grounding, evidence, introspection, surfaces | | gui/ | 20.3k | 20 | PySide6 desktop app | | kernel/ | 14.5k | 8 | the engine (pipeline driver) | | cognition/ | 14.0k | 27 | agent bus, orchestrator, inference, persona, modes | | tools/ | 8.9k | 30 | image engine, news, etc. | | memory/ |

7.0k | 13 | SQLite/FTS5 + FAISS + KG || perception/ | 6.8k | 20 | vision, STT, TTS, OS control || core/ | 6.6k | 24 | paths, settings, hardware profile || planning/ | 3.9k | 24 | proactive daemon, task planning || learning/ | 3.3k | 12 | LoRA self-training || plugins/ | 1.9k | 28 | runtime plugin manager || world/ | 1.5k | 26 | world event bus, local world bridge || integrations/,utils/,contracts/,system/,cli/ | ~5.5k | — | misc |

Four files carry ~1/3 of the codebase: `executor_enhanced.py` (14.3k LOC), `engine.py` (13.4k), `gui/eli_pro_audio_gui_MKI.py` (11.0k), `router_enhanced.py` (7.1k). Next tier: `labs_tab.py` (5.7k), `memory.py` (4.5k), `deterministic_grounding_gate.py` (4.3k).

3. Architecture, layer by layer

- **Boot / hardware** — `core/hardware_profile.py` + `core/startup_hardware_optimizer.py` adapt `n_ctx` / `gpu_layers` / `batch` to whatever model + GPU are present (filename→ctx table; VRAM compute-buffer reservation). Model-agnostic.
- **Routing** — `execution/router_enhanced.py`: regex-first with LLM-intent fallback + an explicit priority pipeline → one of **208 manifest capabilities (183 routable; 174 executor SUPPORTED_ACTIONS)**. Full reference with activation phrases: `capabilities_and_actions.md`.
- **Orchestration** — `kernel/engine.py` gates between the 12-stage cognition/`orchestrator.py` `AgentOrchestrator` (non-quick modes) and the parallel 14-agent cognition/`agent_bus.py` `AgentBus` (quick / fallback). ReAct tool-chaining for non-CHAT actions. Selection now flows through one typed `ExecutionPlan` (`execution/execution_planner.py`). See `orchestration_and_agents.md` for full detail.
- **Inference** — `cognition/gguf_inference.py`: model-path resolved from `env` / `settings` (no baked model); `RLock`-serialized; live runtime override; hot-swap with vision models.
- **Memory** — `memory/memory.py` is the shared foundation (FTS5 + FAISS + KG via `recall_memory`); two deliberate retrieval strategies sit on top (lightweight bus `BusMemoryAgent` vs heavyweight orchestrator `OrchestratorMemoryAgent` with HyDE + RAG + rerank). `memory/knowledge_graph.py`, `memory/vector_store.py` (FAISS, JSON meta).
- **Grounding / evidence (the crown jewel)** — `runtime/deterministic_grounding_gate.py` (4.3k), `runtime/evidence_ledger.py`, `runtime/evidence_arbitration.py`, `runtime/control_contracts`, `runtime/grounded_remediation.py` (1.6k), `cognition/output_governor.py`. A layered, deterministic anti-confabulation system wrapped around the probabilistic model. Most local-LLM projects have nothing comparable.
- **Security** — `runtime/security.py` `SecurityManager`: **fail-closed shell gate** (`ELI_ALLOWED_CMDS` / `ELI_FULL_CONTROL`; unset ⇒ commands blocked), path allow-roots (`ELI_ALLOW_ROOTS`, defaults to project + home), app allowlist with a default-safe set, **SHA-256 custom-agent trust registry** (`config/trusted_agents.json`), prompt-injection guard, SQL identifier validation.
- **Self-model** — `cognition/reasoning_modes.py`: quick/fast/balanced all fastpath to quick, plus four private modes (`chain_of_thought`, `self_consistency`, `tree_of_thoughts`, `constitutional_ai`). Persona overlay, `runtime/self_improvement.py`, LoRA self-training (`learning/`). Emergent internal-state surfacing (autonomy pressure, “anomaly room”) is intentional — see `memory/eli-emergent-voice`.

4. What’s genuinely strong

1. **The grounding / evidence layer.** Unusually rigorous; deterministic evidence gating around a probabilistic model is the right idea and well-developed. This is the part closest to genuinely frontier.
2. **Truly local + truly model-agnostic.** No call-home, hardware-adaptive, swappable models (verified — inference path carries no baked model identity).
3. **Real, fail-closed security.** Shell blocked by default, hash-gated custom agents, path/app allowlists. Most “AI agent” projects ship wide open.
4. **Breadth, integrated.** Vision, voice, OS control, memory, KG, plugins, self-training — all local, all wired into one pipeline.

5. Where it's weak (grounded in the sweep)

1. **2,565 except Exception: blocks (+7 bare except:).** The dominant structural problem. Errors are swallowed into “skipped”/fallback/empty everywhere — which is precisely why bugs surface only via runtime logs. Failures are invisible by design. Frontier software makes failures **loud in dev, quiet in prod**; this swallows them uniformly. Also the main reason the system is hard to reason about. (Only 12 TODO/FIXME markers exist — not because the code is clean, but because failures are absorbed rather than flagged.)
2. **God-files.** Two ~13-14k-line modules (`executor_enhanced.py`, `engine.py`) plus an 11k GUI. The executor is a giant if/elif action ladder. High regression surface, hard to hold in the head, painful to unit-test.
3. **Duplication & overlap.** The shadowed standalone image-engine module (1.75k LOC) was removed, leaving the single `eli/tools/image_engine/` package. `runtime/` (80 files) has many near-duplicate `personal_memory_*` / `*_surface` / `*_response` modules doing overlapping grounding work. Several plan representations coexisted (partly consolidated). Fingerprint of fast solo iteration — new code added beside the old, not folded in.
4. **Repo hygiene.** Committed junk at root: `empty ...` and `[package-index-options]` files, one-off `patch_gpu_dynamic.py` / `patch_sll_bugs.py`, three `verify_eli_claims*.sh` versions, `diag` outputs, `.coverage`, and `experimental/*.zip` binaries. Makes the repo look less serious than the code is.
5. **Tests are GREEN (measured 2026-07-01).** 194 test files; `pytest tests/` = **7,347 passed / 5 failed / 45 skipped / 2 xfailed** (~11m23s on the `.venv/GPU`). The 5 remaining reds are the in-progress `smart_home` plugin removal + one stale blueprint ref — pre-existing and unrelated (they fail identically on a clean tree). The `tests/claims/` contract layer makes it a real safety net.
6. **13 monkeypatch/globals() hacks** — mostly load-bearing (e.g. the CPU-clip vision fix), but they're fragile seams worth tracking.

6. Honest verdict on “frontier, ground-breaking”

In ambition and in specific subsystems — yes. The grounding layer, the fully local model-agnostic design, and the integrated multi-modal local agent are genuinely ahead of most open local-assistant projects. The ideas are frontier-grade.

In engineering discipline — not yet. The 2,565 swallowed exceptions, the god-files, the duplication, and the clutter separate “an extraordinarily ambitious solo project” from “software others can build on.” None of that is a vision problem — it is consolidation and observability. The gap to “ground-breaking” is **subtraction and discipline, not more features.**

7. Highest-leverage work (the next month)

Ranked by effect-per-effort:

1. **Tame error-swallowing.** Replace blanket `except Exception: pass/fallback` with scoped exception types + a single structured error log (a ring buffer / table) you can actually watch. Keep the graceful degradation, but record every swallow. This alone makes the whole system debuggable and is the precondition for trusting any other change.
2. **Split the two god-files** along their natural seams (executor: action groups into per-domain modules behind the dispatch table; engine: pipeline stages into stage modules). Reduces regression surface and makes the pipeline readable.
3. **Delete duplication + clutter.** Collapse the overlapping `runtime/` surfaces, remove root junk/one-off scripts. *(The suite is already green — 7,347 passing; this is now signal-to-noise hygiene, not a red-test cleanup.)*
4. **Consolidate the runtime/ surfaces.** The many `personal_memory_*` / `*_surface` / `*_response` modules want to be a handful of well-named ones.

Do 1-3 and the engineering would match the ideas — which is the only thing standing between ELI and the label “frontier, ground-breaking software people can rely on.”